

Lecture 12: Attention Mechanisms

Week 5, Lecture 1 - From Seq2Seq to Attention

PSYC 51.07: Models of Language and Communication

Winter 2026

Today's Agenda

1.  **The Context Problem:** Why static embeddings aren't enough
2.  **Sequence-to-Sequence Models:** The foundation
3.  **The Bottleneck Problem:** Why vanilla Seq2Seq struggles
4.  **Attention Mechanisms:** The breakthrough innovation
5.  **How Attention Works:** Step-by-step computation
6.  **Visualizing Attention:** Interpreting the weights

Goal: Understand why and how attention revolutionized NLP

Why do we need context-aware models?

Why do we need context-aware models?

Example: The word "bank"

1. "I deposited money at the **bank**" (financial institution)
2. "We sat by the river **bank**" (riverside)
3. "The plane started to **bank** left" (tilt/turn)

Static Embeddings (Word2Vec):

- One vector per word
- **Can't distinguish meanings!**
- Context-independent

Why do we need context-aware models?

Example: The word "bank"

1. "I deposited money at the **bank**" (financial institution)
2. "We sat by the river **bank**" (riverside)
3. "The plane started to **bank** left" (tilt/turn)

Static Embeddings (Word2Vec):

- One vector per word
- **Can't distinguish meanings!**
- Context-independent

Context-Aware Models:

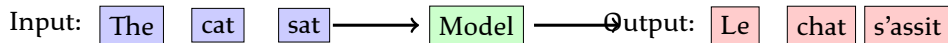
- Different representation per occurrence
- Uses surrounding context
- Handles polysemy naturally

Sequence-to-Sequence Models

The breakthrough for variable-length input/output problems

Applications:

- Machine Translation: English $\rightarrow$ French
- Summarization: Long text $\rightarrow$ Short summary
- Question Answering: Question + Context $\rightarrow$ Answer
- Dialogue Systems: User input $\rightarrow$ System response



Reference: Sutskever et al. (2014) - "Sequence to Sequence Learning with Neural Networks"

Two-part architecture: Encode then Decode

Encoder:

- Reads input sequence
- Compresses to fixed-size vector
- Captures semantic meaning

Decoder:

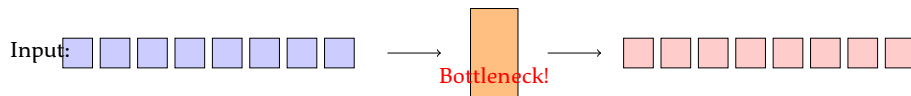
- Starts from context vector
- Generates output sequence
- One token at a time

The Seq2Seq Bottleneck Problem ⚠

Challenge: All information compressed into single vector!

The Problem

- Long sequences → information loss
- Fixed-size context vector is a bottleneck
- Early tokens forgotten by the time we reach the end
- Performance degrades with sequence length

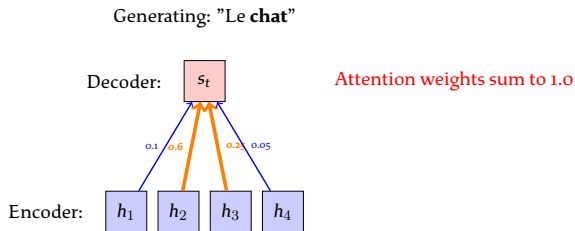


Solution: Attention! ⚡

Attention Mechanism: The Big Idea ⚡

Instead of compressing everything into one vector...

Let the decoder look at all encoder hidden states!



Input: "The cat sat down"

Key Insight: When generating "chat" (cat), pay more attention to "cat" in the input!

Reference: Bahdanau et al. (2015) - "Neural Machine Translation by Jointly Learning to Align and Translate"

How Attention Works: Step by Step

Computing attention weights:

1. **Score:** How relevant is each encoder state to current decoder state?

$$e_{t,i} = \text{score}(s_t, h_i) = s_t^T W_a h_i$$

2. **Normalize:** Convert scores to probabilities (softmax)

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^T \exp(e_{t,j})}$$

3. **Context:** Weighted sum of encoder states

$$c_t = \sum_{i=1}^T \alpha_{t,i} h_i$$

4. **Decode:** Use context vector along with decoder state

$$s_{t+1} = f(s_t, c_t, y_t)$$

Attention Score Functions

Different ways to compute the score

1. Additive Attention (Bahdanau et al. 2015):

$$\text{score}(s_t, h_i) = v^T \tanh(W_1 s_t + W_2 h_i)$$

- Learns alignment jointly with translation
- More parameters, more flexibility

2. Multiplicative Attention (Luong et al. 2015):

$$\text{score}(s_t, h_i) = s_t^T W h_i$$

- Simpler, faster computation
- Works well in practice

3. Scaled Dot-Product (Vaswani et al. 2017):

$$\text{score}(s_t, h_i) = \frac{s_t^T h_i}{\sqrt{d}}$$

- Used in Transformers (coming next lecture!)
- Scaling prevents vanishing gradients

Reference: Luong et al. (2015) - "Effective Approaches to Attention-based Neural Machine Translation"

Example: English → French translation with attention weights

Output (French)	Input Words			
	The	agreement	on	European
L'	orange!800.7	0.1	0.1	0.1
accord	0.1	orange!800.8	0.05	0.05
sur	0.05	0.05	orange!800.8	0.1
l'européen	0.05	0.05	0.1	orange!800.8

Observations:

- Diagonal pattern for similar word order
- Model learns alignment automatically!
- No need for explicit word alignment annotations
- Can handle reordering (syntax differences between languages)

Attention provides interpretability: we can see what the model is "looking at"

Benefits of Attention Mechanisms ✓

1. Solves the Bottleneck Problem

- Decoder has access to all encoder states
- No information compression into single vector
- Works well for long sequences

2. Improves Performance

- Better BLEU scores on translation tasks
- Handles long-range dependencies
- More robust to sequence length

3. Provides Interpretability

- Can visualize what the model focuses on
- Helps debug and understand model behavior
- Builds trust in model predictions

4. Enables Better Alignment

- Learns source-target correspondences
- No need for external alignment tools
- Works across different language pairs

Attention became the foundation for modern NLP!

Real-World Impact of Attention

Attention mechanisms revolutionized multiple domains:

Machine Translation:

- Google Translate (2016)
- DeepL
- Facebook translations
- Dramatic quality improvements

Text Summarization:

- News article summarization
- Document understanding
- Email auto-responses

Question Answering:

- Reading comprehension
- Search engines
- Virtual assistants

Speech Recognition:

- Attend to acoustic features
- Better transcription accuracy
- Listen, Attend and Spell models

Key Milestone

By 2016, attention-based models became the standard for sequence-to-sequence tasks, paving the way for the Transformer revolution in 2017.

Simple attention mechanism implementation

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class BahdanauAttention(nn.Module):
    def __init__(self, hidden_dim):
        super().__init__()
        self.W_dec = nn.Linear(hidden_dim, hidden_dim)
        self.W_enc = nn.Linear(hidden_dim, hidden_dim)
        self.v = nn.Linear(hidden_dim, 1)

    def forward(self, decoder_hidden, encoder_outputs):
        # decoder_hidden: [batch, hidden_dim]
        # encoder_outputs: [batch, seq_len, hidden_dim]
```

Discussion Questions

1. **Why is attention called "soft alignment"?**
 - How is it different from hard alignment?
 - What are the advantages of soft vs. hard?
2. **Computational Cost:**
 - What is the time complexity of attention?
 - How does it scale with sequence length?
 - When might this be a problem?
3. **Interpretability:**
 - Can we always trust attention weights as explanations?
 - What about when attention is uniform across all inputs?
4. **Beyond Seq2Seq:**
 - Where else might attention be useful?
 - Can we apply it within a single sequence?
 - (Spoiler: Yes! That's self-attention ☐ next lecture!)

What's Next?

Today we learned:

- The context problem in NLP
- Sequence-to-sequence architecture
- The bottleneck problem
- How attention mechanisms work
- Attention as alignment and interpretation

Next lecture (Lecture 13):

- **Self-Attention**: Attention within a sequence
- **The Transformer**: "Attention is All You Need"
- **Query, Key, Value**: The new framework
- **Multi-Head Attention**: Learning diverse relationships
- **Positional Encoding**: Injecting order information

Get ready for the Transformer revolution! 

Key Takeaways:

1. Context Matters

- Static embeddings can't capture context-dependent meanings
- Need dynamic representations based on context

2. Seq2Seq Bottleneck

- Fixed-size context vector limits performance
- Information loss for long sequences

3. Attention is the Solution

- Dynamic access to all encoder states
- Weighted combination based on relevance
- Attention weights sum to 1.0 (probability distribution)

4. Benefits

- Better performance on long sequences
- Automatic alignment learning
- Model interpretability

Attention mechanisms laid the foundation for modern transformer-based models!

Key Papers:

- **Sutskever et al. (2014)** - "Sequence to Sequence Learning with Neural Networks"
 - Introduced encoder-decoder architecture
 - Foundation for seq2seq models
- **Bahdanau et al. (2015)** - "Neural Machine Translation by Jointly Learning to Align and Translate"
 - Introduced additive attention mechanism
 - Solved the bottleneck problem
- **Luong et al. (2015)** - "Effective Approaches to Attention-based Neural Machine Translation"
 - Multiplicative attention variants
 - Global vs. local attention
- **Vaswani et al. (2017)** - "Attention Is All You Need"
 - The Transformer architecture (next lecture!)
 - Scaled dot-product attention

Additional Resources:

- Jay Alammar's blog: "Visualizing A Neural Machine Translation Model"
- Distill.pub: "Attention and Augmented Recurrent Neural Networks"

Discussion Time

Office Hours Topics:

- Implementing attention from scratch
- Different attention mechanisms
- Debugging attention-based models
- Assignment 4 preparation

Thank you! 

See you next lecture for Transformers!